Chapter 6

Getting Started with Node.js with AI Assistance

Table of Contents

Chapter 6 1

Getting Started with Node.js 1

6.1. Introduction 2

6.2. Streams and buffers 4

6.2.1. Writing stream 9

6.2.2. pipe 11

6.3. Fundamentals of Node.js 14

6.3.1. Global object 16

6.3.2. Function expression in NodeJS 18

6.3.3. module 'require' 18

6.3.4. Events and Util Modules in NodeJS 21

6.3.5. Reading and writing files in nodejs 22

6.3.6. Create and remove files 27

6.4. Clients and servers connection using nodeJS 31

6.4.1. How the Client-Server Model works ? 31

6.4.2. How to create a server in nodejs 33

6.4.3. Serving HTML 34

6.3.4. Serving JSON data 35

6.4.5. Routing in node.js 36

References 43

6.1. Introduction

Node.js is an open-source, cross-platform, JavaScript runtime environment that executes JavaScript code outside of a web browser. Node.js is a popular, lightweight web framework for beginners, and it is used by many big companies like Netflix and Uber.

When we typically think of JavaScript, our mind tends to go to web development. Until Node.js came along, there was really no way to run JavaScript outside of a browser. When we write a backend server and database, Node.js is a popular choice because we can run our code as a standalone application rather than something that can only be evaluated in a browser environment. Node.js is an important tool for any JavaScript developer to understand. (Educative, 2023)

What is Node.js?

Node.js is an open-source, cross-platform JavaScript runtime environment used for executing JavaScript code outside of a web browser.

Node.js is a great web framework for beginners because it works great for **data-intensive applications**, like streaming and real-time apps, and Node.js makes it easy to start **building the back-end**.

Node.js allows us to **use JavaScript everywhere** and on any browser, including MacOS, Linux, and Windows. When we say everywhere, we mean the front-end, the middle-ware, and the back-end. Node.js is, therefore, part of some very popular web development stacks, such as the MERN stack, MEVN stack, and MEAN stack.

There are a number of characteristics that make Node.js what it is:

- **Google Chrome V8 JavaScript Engine:** This runtime environment is built on the Google Chrome V8 JavaScript runtime engine. In the same way a Java Virtual Machine translates bytecode, the Chrome V8 JavaScript engine takes JavaScript and makes it readable.
- **Modules/Packages:** Node.js has npm, a node package manager, with a library of over 350,000 packages to help get your project or application off the ground with efficiency and ease.
- **Event Driven, Single-Threaded I/O Model:** JavaScript relies on user interactions or events to run. In most cases, code is run synchronously. Server requests and other such asynchronous tasks rely on a system of promises or async/await functions to handle these inputs and outputs. (w3schools, n.d.)

Why Node.js?

Node.js uses asynchronous programming!

A common task for a web server can be to open a file on the server and return the content to the client.

Here is how PHP or ASP handles a file request:

1. Sends the task to the computer's file system.
2. Waits while the file system opens and reads the file.
3. Returns the content to the client.
4. Ready to handle the next request.

Here is how Node.js handles a file request:

1. Sends the task to the computer's file system.
2. Ready to handle the next request.
3. When the file system has opened and read the file, the server returns the content to the client.

Node.js eliminates the waiting, and simply continues with the next request.

Node.js runs single-threaded, non-blocking, asynchronous programming, which is very memory efficient.

What Can Node.js Do?

- Node.js can generate dynamic page content.
- Node.js can create, open, read, write, delete, and close files on the server.
- Node.js can collect form data.
- Node.js can add, delete, modify data in your database.

What is a Node.js File?

- Node.js files contain tasks that will be executed on certain events
- A typical event is someone trying to access a port on the server
- Node.js files must be initiated on the server before having any effect
- Node.js files have extension ".js"

6.2. Streams and buffers

To handle and manipulate streaming data like a video, a large file, etc., we need streams in Node. The streams module in Node.js manages all streams. Stream is data that flows from one place to another.

Types of Streams

In Node, there are four different types of streams:

- Readable streams → To create a stream of data for reading (say, reading a large file in chunks).
- Writable streams → To create a stream of data for writing (say, writing a large amount of data to a file).
- Duplex streams → To create a stream that is both readable and writable at the same time. We can read and write to a duplex stream (say, a socket connection between a client and a server).
- Transform streams → To create a stream that is readable and writable, but the data can be modified while reading and writing to the stream (say, compressing data by the client and server before while requesting).

Buffers in Streams

Streams work on a concept called *buffer*.
A *buffer* is a temporary memory that a stream takes to hold some data until it is consumed.

In a stream, the buffer size is decided by the `highWatermark` property on the stream instance which is a number denoting the size of the buffer in bytes.

A buffer memory in Node by default works on `String` and `Buffer`**.** We can also make the buffer memory work on JavaScript objects. To do so, we need to set the property `objectMode` on the stream object to `true`**.**

If we try to push some data into the stream, the data is pushed into the stream buffer. The pushed data sits in the buffer until the data is consumed.
If the buffer is full and we try to push data to a stream, the stream does not accept that data and returns with a false value for the push action.

Readable stream

readable stream allows node.js to read data from a stream. A stream that is used to read the streaming data is called a *readStream()*. A *readStream()* can be reading a file from a server, or streaming an online video.

An analogy that I found in many places is to think of a readable stream like a faucet, and the analogy fits perfectly for this stream. The faucet will have water *(or, the data)* passing through it that is being consumed by someone.

To create a readable stream we use the ***createReadStream()*** method. (Tandon, 2019)

Example 1) create a read stream from a large text file:

app.js readme.txt

readme.txt

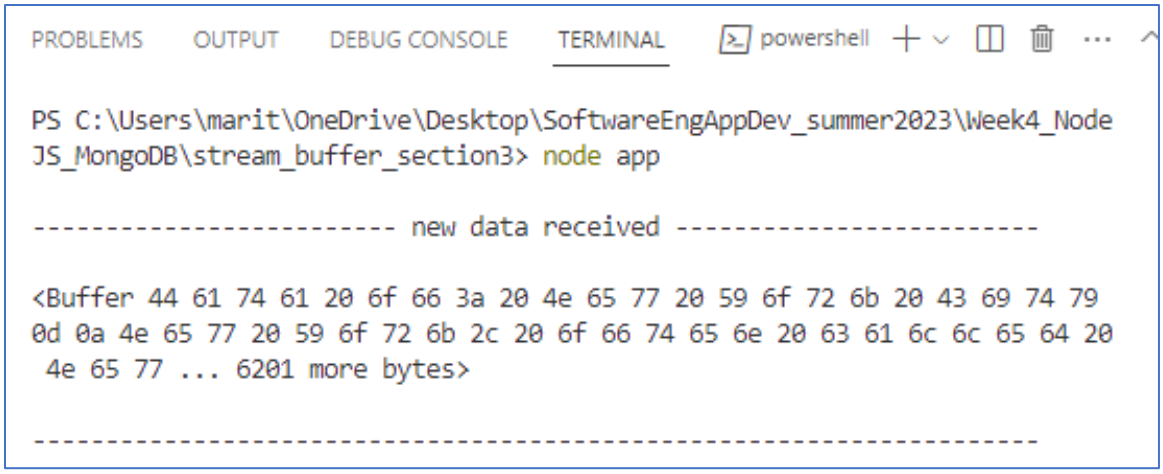
```
1  Data of: New York City
2  New York, often called New York City[a] or NYC, is the most populous city in the United States.
   With a 2020 population of 8,804,190 distributed over 300.46 square miles (778.2 km2), New York City
   is the most densely populated major city in the United States. The city is more than twice as
   populous as Los Angeles, the nation's second-largest city. New York City is situated at the
   southern tip of New York State. Situated on one of the world's largest natural harbors, New York
   City comprises five boroughs, each of which is coextensive with a respective county. The five
   boroughs, which were created in 1898 when local governments were consolidated into a single
   municipality, are: Brooklyn (Kings County), Queens (Queens County), Manhattan (New York County),
   the Bronx (Bronx County), and Staten Island (Richmond County).[11] New York City is a global city
   and a cultural, financial, high-tech,[12] entertainment, glamor,[13] and media center with a
   significant influence on commerce, health care and scientific output in life sciences,[14][15]
   research, technology, education, politics, tourism, dining, art, fashion, and sports. Home to the
   headquarters of the United Nations, New York is an important center for international diplomacy,[16]
   [17] and it is sometimes described as the world's most important city[18] and the capital of the
   world.[19][20]
3  The city is the geographical and demographic center of both the Northeast megalopolis and the New
   York metropolitan area, the largest metropolitan area in the U.S. by both population and urban
   area. With over 20.1 million people in its metropolitan statistical area and 23.5 million in its
   combined statistical area as of 2020, New York City is one of the world's most populous megacities.
   [21] The city and its metropolitan area are the premier gateway for legal immigration to the United
   States. As many as 800 languages are spoken in New York,[22] making it the most linguistically
   diverse city in the world. New York City is home to more than 3.2 million residents born outside
   the U.S., the largest foreign-born population of any city in the world as of 2016.[23] It is the
   most visited U.S. city by international visitors.[24] Providing continuous 24/7 service and
   contributing to the nickname The City That Never Sleeps, the New York City Subway is the largest
   single-operator rapid transit system in the world with 472 passenger rail stations, and Penn
   Station in Midtown Manhattan is the busiest transportation hub in the Western Hemisphere.[25].
4  New York City traces its origins to Fort Amsterdam and a trading post founded on the southern tip
   of Manhattan Island by Dutch colonists in approximately 1624. The settlement was named New
   Amsterdam (Dutch: Nieuw Amsterdam) in 1626 and was chartered as a city in 1653. The city came under
   British control in 1664 and was renamed New York after King Charles II granted the lands to his
   brother, the Duke of York.[26] The city was temporarily regained by the Dutch in July 1673 and was
   renamed New Orange; the city has been named New York since November 1674. New York City was the
   capital of the United States from 1785 until 1790,[27] and has been the largest U.S. city since
   1790. The Statue of Liberty greeted millions of immigrants as they came to the U.S. via Ellis
   Island by ship in the late 19th and early 20th centuries, and is a symbol of the U.S. and its
```

app.js readme.txt

app.js > ...

```
1   const http = require('http')
2   const fs = require('fs')
3
4   //create a readable stream to read file readme.txt. This will fill up the buffer
5   const readStream = fs.createReadStream(__dirname + '/readme.txt')
6
7   //listen to the stream: buffer passes the data through the stream and recognize it as 'data'
8   readStream.on('data',(d)=>{
9       console.log('\n------------------------- new data received -------------------------\n ')
10      console.log(d)
11      console.log('\n---------------------------------------------------------------------\n ')
12  })
```

When executed, the following code will give the output as:



We get the buffer data, which is nothing but the byte data for the content that is in the buffer memory of the stream.

We can also have an open or close on the data stream:

```
const http = require('http')
const fs = require('fs')

//create a readable stream to read file readme.txt. This will fill up the buffer
const readStream = fs.createReadStream(__dirname + '/readme.txt')

//listen to the stream: buffer passes the data through the stream and recognize it as 'data'
readStream.on('data', (d) => {
    console.log('\n------------------------- new data received ------------------------\n ')
    console.log(d)
    console.log('\n--------------------------------------------------------------------\n ')
})

readStream.on('open', () => {
    console.log('\n\nStream opened...');
});

readStream.on('end', () => {
    console.log('Stream Closed...\n\n');
});
```

When executed, we will see the open and end stream in the terminal:

```
Stream opened...

------------------------- new data received -------------------------

<Buffer 44 61 74 61 20 6f 66 3a 20 4e 65 77 20 59 6f 72 6b 20 43 69 74 79
 0d 0a 4e 65 77 20 59 6f 72 6b 2c 20 6f 66 74 65 6e 20 63 61 6c 6c 65 64
20 4e 65 77 ... 6201 more bytes>

---------------------------------------------------------------------

Stream Closed...
```

Pause and resume a readable stream

You can also pause and resume a stream in Node.js by simply calling the `pause()` and `resume()` function on the stream.

As a result of calling the `pause()` function, the data event does not get triggered until we call the `resume()` function of the stream.

Flowing and Non-Flowing Streams

There are two types of readable streams:

- Flowing stream — A stream that keeps on passing the data that can be directly listened to by using the data event on the stream.
- Non-flowing stream — A stream that does not push data automatically. Instead, the stream stores the data in the buffer and we need to explicitly call the read() method of the stream to read it.

The above code was an example of a flowing stream as we were just listening to the `data` event of the stream and the listener gets triggered automatically every time some new data comes up from the stream.

The following is a simple example of a non-flowing stream:

```
// flowing stream: read 10 == code doesnt work
setTimeout(()=>{
    const data = readStream.read(10)
    console.log('\n------------- read 10 data ------------- ')
    console.log(data)
},1000)
```

When running the code, it will give the following output:

```
<Buffer 4c 6f 72 65 6d 20 69 70 73 75>
```

But how did it work?

It's because we created a read stream using the `createReadStream` method from the FS module.
As soon as the stream gets created, the file data will start streaming into the stream variable. We also allowed some time for the stream, using the `setTimeout` method, to get some data filled in its buffer.

After 10 milliseconds, the `setTimeout` callback gets executed and we read the first 10 bytes of the buffer using the `read()` method called with 10 (size in bytes) as an argument.

Buffer Management by the Read Stream

In the above code, if we call the `read()` function again after the `console.log(data)` and print the new data, we see that the data is different from the previous log:

```
import { createReadStream, ReadStream } from 'fs';
let readStream: ReadStream = createReadStream('./data.txt');
setTimeout(() => {
     const data = readStream.read(10);
     console.log(data);
     const data2 = readStream.read(10);
     console.log(data2);
}, 10);
```

6.2.1. Writing stream

The **createWriteStream() method** is an inbuilt application programming interface of fs module which allows to quickly make a writable stream for the purpose of writing data to a file. This method may be a smarter option compared to methods like fs.writeFile when it comes to very large amounts of data.

Syntax:

```
fs.createWriteStream( path, options )
```

Parameters: This method accept two parameters as mentioned above and described below:

- **path:** This parameter holds the path of the file where to read the file. It can be String, Buffer or URL.
- **options:** It is an optional parameter that holds string or object.

Return Value: This method returns the **fs.writeStream** object.
(nitrin_sharma, n.d.)

Example 2)

```
const writestream = fs.createWriteStream('write.txt')
writestream.write('Streaming text content','utf-8')
```

The *write.txt* file will be created with the content as:

```
Streaming text content
```

We can also stream data of a file into a new file using the ***write*** method in ***createWriteStream()***

write.txt app.js

app.js > ...

```
1   const http = require('http')
2   const fs = require('fs')
3
4   //WRITE STREAM
5   const writestream = fs.createWriteStream('write.txt')
6   writestream.write('writestream','utf-8')
7
8   //READ STREAM
9   //create a readable stream to read file readme.txt. This will fill up the buffer
10  const readStream = fs.createReadStream(__dirname + '/readme.txt')
11
12  //listen to the stream: buffer passes the data through the stream and recognize it as 'data'
13  readStream.on('data', (d) => {
14      console.log('\n------------------------- new data received -------------------------\n ')
15      console.log(d)
16      console.log('\n---------------------------------------------------------------------\n ')
17      writestream.write(d)
18  })
```

We can have some text in the *write.txt* file as:

write.txt app.js

write.txt

```
writestreamData of: New York City
New York, often called New York City[a] or NYC, is the most populous city in the United States. With a 2020 population of 8,804,190 distributed over 300.46 square miles (778.2 km2), New York City is the most densely populated major city in the United States. The city is more than twice as populous as Los Angeles, the nation's second-largest city. New York City is situated at the southern tip of New York State. Situated on one of the world's largest natural harbors, New York City comprises five boroughs, each of which is coextensive with a respective county. The five boroughs, which were created in 1898 when local governments were consolidated into a single municipality, are: Brooklyn (Kings County), Queens (Queens County), Manhattan (New York County), the Bronx (Bronx County), and Staten Island (Richmond County).[11] New York City is a global city and a cultural, financial, high-tech,[12] entertainment, glamor,[13] and media center with a significant influence on commerce, health care and scientific output in life sciences,[14][15] research, technology, education, politics, tourism, dining, art, fashion, and sports. Home to the headquarters of the United Nations, New York is an important center for international diplomacy,[16][17] and it is sometimes described as the world's most important city[18] and the capital of the world.[19][20]
The city is the geographical and demographic center of both the Northeast megalopolis and the New York metropolitan area, the largest metropolitan area in the U.S. by both population and urban area. With over 20.1 million people in its metropolitan statistical area and 23.5 million in its combined statistical area as of 2020, New York City is one of the world's most populous megacities.[21] The city and its metropolitan area are the premier gateway for legal immigration to the United States. As many as 800 languages are spoken in New York,[22] making it the most linguistically diverse city in the world. New York City is home to more than 3.2 million residents born outside the U.S., the largest foreign-born population of any city in the world as of 2016.[23] It is the most visited U.S. city by international visitors.[24] Providing continuous 24/7 service and contributing to the nickname The City That Never Sleeps, the New York City Subway is the largest single-operator rapid transit system in the world with 472 passenger rail stations, and Penn Station in Midtown Manhattan is the busiest transportation hub in the Western Hemisphere.[25].
```

6.2.2. pipe

Piping in NodeJS is the process by which byte data from one stream is sent to another stream.

Streams in NodeJS are a sequence of bytes. When a program processes some data it needs to read all the data into memory and then process it and write out the data back into some location. However, this approach fails when there is a large amount of data to be processed. To overcome this problem streaming algorithms are used for processing the data in which the data is not read all at once instead it 'flows' into the program, is processed, and 'flows' out of the program. Here the data is processed in small chunks and the whole data is never held in memory at once. The data stream can be compared with the example of water flowing in a river where a large amount of water is transported by flowing through the river over time. Streaming algorithms are more memory efficient and faster. You can read this article to get a detailed guide on streams.

Piping in NodeJS is useful when we need to write some data coming from a source stream to another stream. In such a case we can use the piping algorithms provided by NodeJS instead of writing our own reading and writing logic. We basically pipe the read stream and write stream to one another. This can be compared with joining two water bodies using a pipe to transfer water.

Let's take an example on piping, suppose you want to write a server that serves some files. In this case, you will need to write code to handle the reading operation on the files and code to write the data to a network socket. But with the piping facility provided by NodeJS, you can easily connect the file stream to the network socket stream to send the data. There is a pipe() function in NodeJS to handle this operation which is provided with a stream object and accepts a stream object.
Pipe function can be used to connect an input stream to an output stream or it can be included with a combination of streams to create a complex pipeline to execute a certain task. (moonpatel2003, 2023)

Syntan:

```
fs.createReadStream('readme.txt').pipe(fs.createWriteStream)
```

Example 3)

```
//READ STREAM
const readStream = fs.createReadStream(__dirname + '/readme.txt')

//WRITE STREAM
const writestream = fs.createWriteStream('write.txt')

//PIPING
readStream.pipe(writestream)
```

we can also pipe the stream data in a browser:

app.js

```
1   const http = require('http')
2   const fs = require('fs')
3
4
5   const server = http.createServer((req, res) => {
6       res.writeHead(200, { 'Content-Type': 'text/plain' })
7       const readStream = fs.createReadStream(__dirname + '/readme.txt')
8       //PIPING
9       readStream.pipe(res)
10
11  })
12
13  server.listen((3000), () => {
14      console.log('serve is running...')
15  })
```

The browser will display:



6.3. Fundamentals of Node.js

Now that we know what Node.js is, let's explore the fundamentals of this tool.

Console

The **console** is a module provided by Node.js that is akin to the JavaScript console in the browser when you inspect a webpage. The console has methods that are available for us to use for debugging purposes.

- `console.log()`: Frequently used to log some sort of output.
- `console.warn()`: Explicitly delivers a warning to the console.
- `console.error()`: Explicitly delivers an error message to the console. You can log an error as a string or as an object. If logged as a new `Error`(), a traceback will be included as part of the message.
- `console.trace()`: Logs a traceback when an error occurs in your code. Gives line number and column number of the file that the error probably occurred.

Example 4) Let's create a file named **app.js** and log "Hello NodeJS" to the console:

```
console.log('Hello NodeJS')
```

We can run this Node.js file from the terminal by using the node command followed by the file name (without the .js extension). For example:

node app

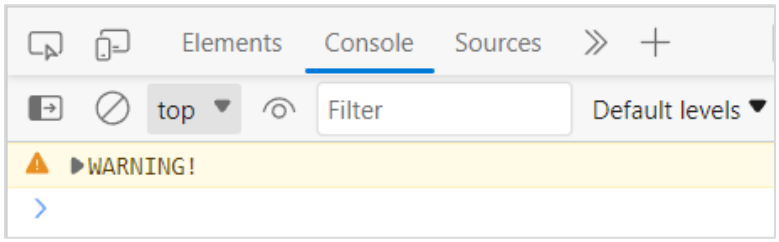
This will execute the app.js file as a Node.js application and display the output in the terminal.



The **console.warn** will display a warning message on the console:

```
console.warn('WARNING!')
```

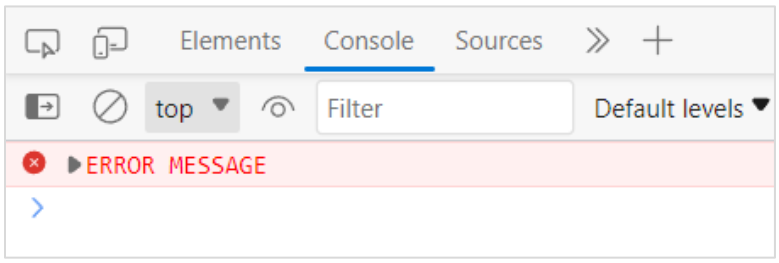
In the browser console will show:



console.error delivers an error message to the console:

```
console.error('ERROR MESSAGE')
```

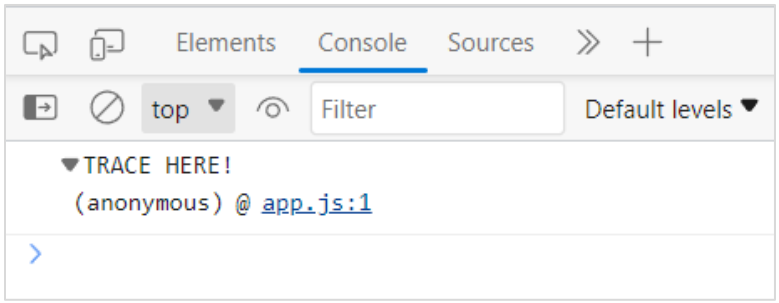
In the browser console will show:



console.trace tracebacks when an error occurs in your code:

```
console.trace('TRACE HERE!')
```

In the browser console will show:



6.3.1. Global object

Global objects are available in every module, so they can be used without importing a specific module. The `Buffer` class, for example, class is defined as a global in Node.js. Some other common global objects are:

- The `console` object is used to print to `stdout` and `stderr`.
- Timers, such as `setImmediate`, `setInterval`, and `setTimeout`, are also globals.
- The `process` object is also global.

In a browser, the top-level scope is the global scope. But in Node.js, the top-level scope is not the global scope. In Node.js, the global object can be used to see whatever is available in the **global scope**. Run the code below to see an example.

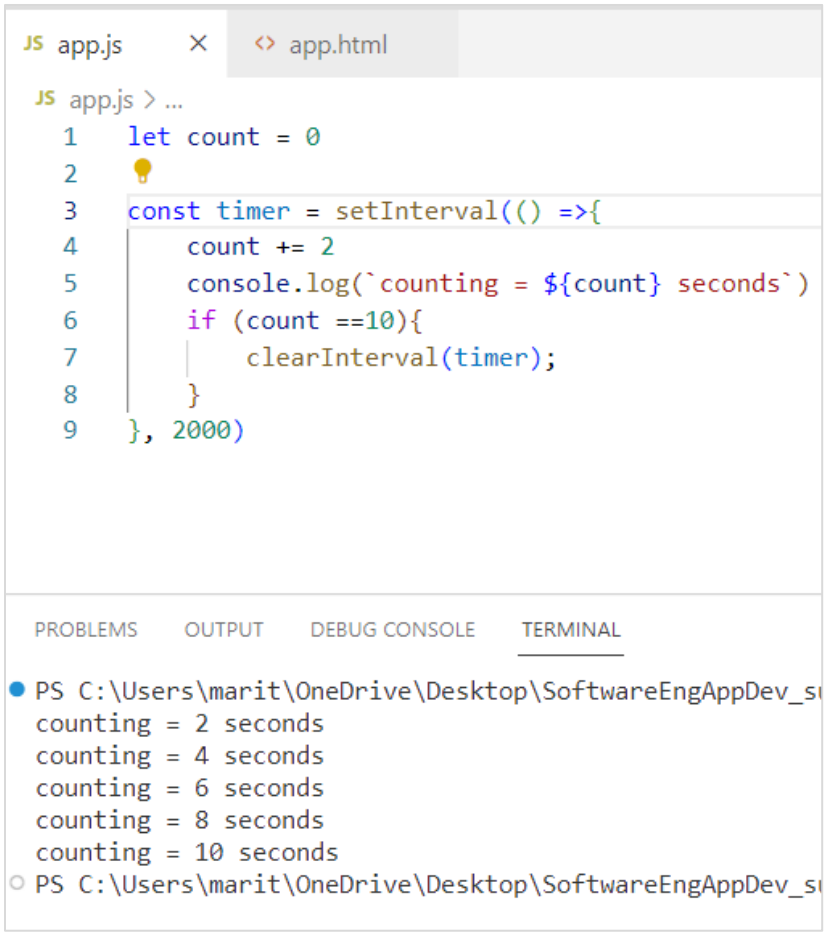
Example 5) use **setTimeout** to display a message in the console in 3 seconds:

app.js × app.html

app.js > ...

```
1   setTimeout(() =>{
2       console.log('Welcome to nodejs')
3   }, 3000)
```

We can also have a time interval to display a counter, from 0 to 10 with an increment of 2 in every 2 seconds



We can also check the current directory that we are working on by typing double underscore follow by the keyword ***dirname***

`__dirname`

We can also check the filename

`__filename`

(Geeks, 2023)

Double underscore (__) in front of a variable is a convention. It is used for the **global variables** (The following variables may appear to be global but are not, rather local to each module) in Nodejs meanwhile **Underscore(_)** used to define a **private variable**.

6.3.2. Function expression in NodeJS

Defining a function in node.js is almost the same defining function in JS.
A function expression is very similar to, and has almost the same syntax as, a function declaration. The main difference between a function expression and a function declaration is the *function name*, which can be omitted in function expressions to create *anonymous* functions. A function expression can be used as an IIFE (Immediately Invoked Function Expression) which runs as soon as it is defined.

Example 6) review three main function structure: variable function, arrow function, and invoke a function within a function

```
let color = function(c){
    console.log(`Color is = ${c}`)
}
color('orange')
```

or we can use an arrow function

```
let color = (c)=>{
    console.log(`Color is = ${c}`)
}
color('orange')
```

we can also invoke a function within a function

```
function callColor(innerfunction){
    innerfunction()
}
let color = function(){
    console.log('Passing a function')
}
callColor(color)
```

6.3.3. module 'require'

Consider modules to be the same as JavaScript libraries. Therefore, a module is basically a set of functions you want to include in your application.

Example 7) let's create another file named ***mod.js*** and make the file accessible for any files to use the content of it. To do so, we use the keyword `require('./ ')` in the file that we want to import the information.

App.js

```
const name = require("./mod")
console.log(name("peter"))
```

In the exported file, we add the keyword ***module.exports = component to be exported.***

mod.js

```
const helper = function(data){
    return `${data} is logged in!`
}
module.exports = helper
```

Running the ***app.js*** file by typing *node app*, the Terminal will prompt:

```
Peter is logged in!
```

Module can also return multiple functions:

```
const helper = function(data){
    return `${data} is logged in!`
}

const id = function(i){
    return `${i}`
}

let email = function(e){
    return e
}
```

Since ***module.exports*** is an object, we can add properties to the object:

```
module.exports.helper = helper
module.exports.id = id
module.exports.email = email
```

We can also make a one-line object to export multiple functions:

```
module.exports = {
    helper, id, email
}
```

Now, when we call the export function in the ***app.js*** file we just add the exported property next to the object:

```
const name = require("./mod")
console.log(name.helper("Peter"))
console.log(name.id(12345))
console.log(name.email("peter@neverland.edu"))
```

Running **node app** in the Terminal will prompt:

```
Peter is logged in!
12345
peter@neverland.edu
```

Event emitter

Node.js is perfect for event-driven applications. Node.js has a built-in module, called "events", where you can create-, fire-, and listen for- your own events.

To include the built-in Events module use the `require()` method. In addition, all event properties and methods are an instance of an `EventEmitter` object. To be able to access these properties and methods, create an `EventEmitter` object.

The event module is used when it toggles an event in the app. When we import an event module into a file, we only type the module name without a path indicator ./

```
let events = require('events');
let eventEmitter = new events.EventEmitter();
```

For our example, let's us create a simple event module to display a message EVENTS IN NODEJS in the console:

```
const events =require('events')

const eventEmitter = new events.EventEmitter

eventEmitter.on('test', function(a){
    console.log(a)
})

eventEmitter.emit('test','EVENTS IN NODEJS')
```

Running *node events.js*, the Terminal will prompt:

```
EVENTS IN NODEJS
```

6.3.4. Events and Util Modules in NodeJS

Let's create a program that will use inherit property of events to retry information from an event emitter.

```
const events =require('events')
const util = require('util')

const teams = function(name){
    this.name = name
}
// eventEmitter will inherent any teams construct
util.inherits(teams, events.EventEmitter)

const Barcelona = new teams('Barcelona')
const Milan = new teams('Milan')
```

```
// save each constructor in an array
const teamsArray = [Barcelona, Milan]

// print each team using forEach loop
teamsArray.forEach((t)=>{
    t.on('nation', function(n){
        console.log(`${t.name} is ${n} football club!`)
    })
})

Milan.emit('nation', 'Italian')
```

6.3.5. Reading and writing files in nodejs

You can use the **fs, file system,** module to read and write files in Node.js.
The **fs** module is a built-in module in Node.js that provides both asynchronous and synchronous functions to read, write, and watch files. (Atta, 2023)

Synchronous methods

The **fs.readFileSync() method** is an inbuilt application programming interface of the **fs** module which is used to read the file and return its content.

Example 8) create an app to read a file named *readme.txt*:

```
Date: April 2023
Message: Read me please!
Author: Prof. Wu
```

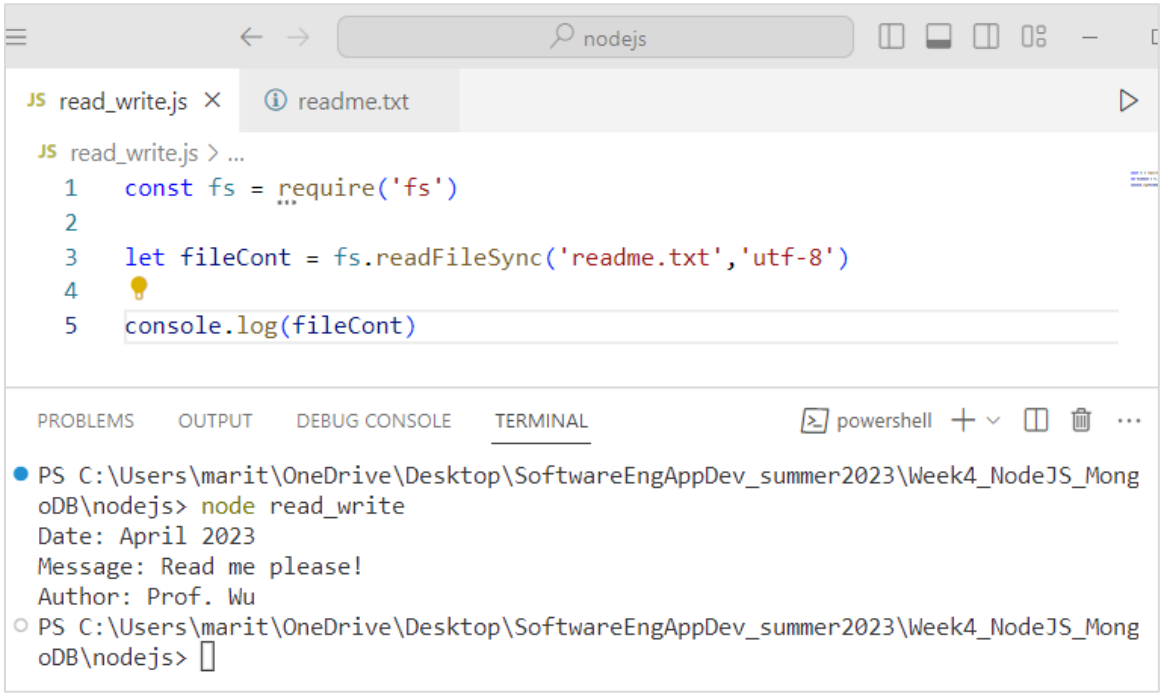
The fs.readFileSync() method is called the original node program stops executing, and the node waits for the fs.readFileSync() function to get executed, after getting the result of the method the remaining node program is executed. (Arnubop, 2023)

Syntax:

`fs.readFileSync( path, options )`

- **path:** It takes the relative path of the text file. The path can be of URL type. The file can also be a file descriptor. If both the files are in the same folder just give the filename in quotes.

- **options:** It is an optional parameter that contains the encoding and flag, the encoding contains data specification. Its default value is null which returns the raw buffer and the flag contains an indication of operations in the file. Its default value is 'r'.



The fs.writeFileSync() is a synchronous method. The fs.writeFileSync() creates a new file if the specified file does not exist. Also, the 'readline-sync' module is used to enable user input at runtime.

Syntax:

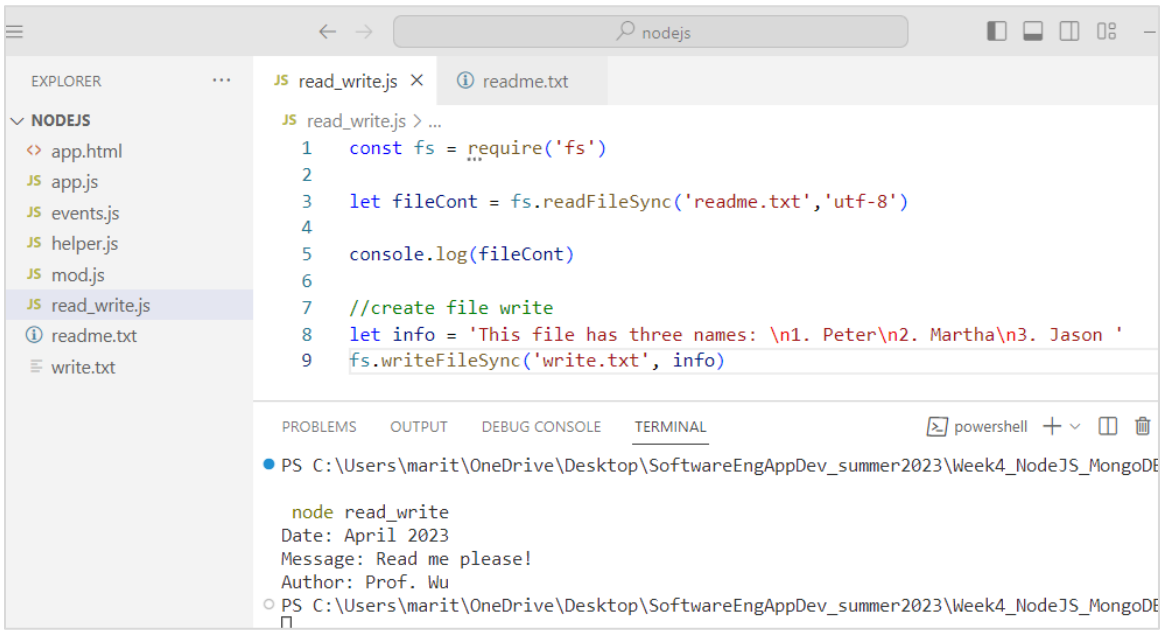
`fs.writeFileSync( file, data, options )`

Parameters: This method accepts three parameters as mentioned above and described below:

- **file:** It is a string, Buffer, URL, or file description integer that denotes the path of the file where it has to be written. Using a file descriptor will make it behave similarly to the fs.write() method.
- **data:** It is a string, Buffer, TypedArray, or DataView that will be written to the file.
- **options:** It is a string or object that can be used to specify optional parameters that will affect the output. It has three optional parameters:

- encoding: It is a string that specifies the encoding of the file. The default value is ‘utf8’.
- mode: It is an integer that specifies the file mode. The default value is 0o666.
- flag: It is a string that specifies the flag used while writing to the file. The default value is ‘w’.

Example 9)



The file *write.txt* was automatically created:

```
This file has three names:
   1. Peter
   2. Martha
   3. Jason
```

Asynchronous functions

The Asynchronous function has a callback function as the last parameter which indicates the completion of the asynchronous function. Node.js developers prefer asynchronous methods over synchronous methods as asynchronous methods never block a program during its execution, whereas the latter does.

Read data from files

The simplest way to read a file in Node.js is to use the **`fs.readFile( )`** method by passing the file path, encoding, and a callback function.

It asynchronously reads the entire contents of the file and calls the callback function with the file data.

Syntax:

```
fs.readFile( filename, encoding, callback_function )
```

Parameters: The method accept three parameters as mentioned above and described below:

- **filename:** It holds the name of the file to read or the entire path if stored at other location.
- **encoding:** It holds the encoding of file. Its default value is **'utf-8'**.
- **callback_function:** It is a callback function that is called after reading of file. It takes two parameters:
    - **err:** If any error occurred.
    - **data:** Contents of the file.

Return Value: It returns the contents/data stored in file or error if any.

Example 10) write a nodejs code that will read the file 'readmore.txt'.
The content of 'readmore.txt' is:

```
Date: July 2024
Message: Please read more about nodejs
Author: student's name
```

The running file should read the 'readmore.txt' as:

```
const fs = require('fs')
fs.readFile('readmore.txt', 'utf-8', function(error,data){
    console.log(data)
    console.log(`Error = ${error}`)
})
```

Write data to files

The **fs.writeFile()** method is used to asynchronously write the specified data to a file. By default, the file would be replaced if it exists. The 'options' parameter can be used to modify the functionality of the method.

Syntax:

```
fs.writeFile( file, data, options, callback )
```

Parameters: This method accept four parameters as mentioned above and described below:

- **file:** It is a string, Buffer, URL or file description integer that denotes the path of the file where it has to be written. Using a file descriptor will make it behave similar to fs.write() method.
- **data:** It is a string, Buffer, TypedArray or DataView that will be written to the file.
- **options:** It is a string or object that can be used to specify optional parameters that will affect the output. It has three optional parameter:
    - **encoding:** It is a string value that specifies the encoding of the file. The default value is 'utf-8'.
    - **mode:** It is an integer value that specifies the file mode. The default value is 0o666.
    - **flag:** It is a string value that specifies the flag used while writing to the file. The default value is 'w'.
- **callback:** It is the function that would be called when the method is executed.
    - **err:** It is an error that would be thrown if the operation fails.

Example 11) write the 'info' string into the 'readmore.txt' file from the previous example

```
let info = 'This file has three names: \n1. Peter\n2. Martha\n3. 
Jason '
fs.writeFile('readmore.txt',info, function(error){
    if (error){
        console.log(error)
    }
})
```

When click on 'readmore.txt' file, the previous information gets overwritten:

```
This file has three names:
        1. Peter
        2. Martha
        3. Jason
```

Appending content to a file

Appending to files is handy when you don't want to overwrite a file with new content, but rather add to it. A handy method to append content to the end of a file is `fs.appendFile()` (and its `fs.appendFileSync()` counterpart):

Example 9) Add two more names to the previous example file 'readmore.txt'

```
let extraInfo = '\n4. George\n5. Ann'
fs.appendFile('readmore.txt', extraInfo, function(error){
    if (error){
        console.log(error)
    }
})
```

The `readmore.txt` file will look as:

```
This file has three names:
        1. Peter
        2. Martha
        3. Jason
        4. Goerge
        5. Ann
```

6.3.6. Create and remove files

Remove files

The **fs.unlink() method** is used to remove a file or symbolic link from the filesystem. This function does not work on directories, therefore it is recommended to use fs.rmdir() to remove a directory.

Syntax:

fs.unlink(path, callback)

Parameters: This method accepts two parameters as mentioned above and described below:

- **path:** It is a string, Buffer or URL which represents the file or symbolic link which has to be removed.
- **callback:** It is a function that would be called when the method is executed.
- **err:** It is an error that would be thrown if the method fails.

Example 12) remove the file '**write.txt**'

```
fs.unlink('write.txt',(error)=>{
    console.log(error)
})
```

The **fs.unlinkSync() method** is used to synchronously remove a file or symbolic link from the filesystem. This function does not work on directories, therefore it is recommended to use fs.rmdir() to remove a directory.

Syntax:
fs.unlinkSync(path)

Parameters: This method accepts one parameter as mentioned above and described below:

- **path:** It is a string, Buffer or URL which represents the file or symbolic link which has to be removed.

Example 13) delete file 'readmore.txt' from current directory

```
fs.unlinkSync('readme.txt')
```

Creating files

Creating a file in Node.js is straightforward thanks to the `fs` (file system) module, which provides various methods to interact with the file system. Here are a few common ways to create a file in Node.js:

- Using `fs.writeFile` or `fs.writeFileSync`
- Using `fs.appendFile` or `fs.appendFileSync`
- Using `fs.open` and `fs.write` or `fs.openSync` and `fs.writeSync`

Example 14) Create a new file named 'example.txt' and write a message 'Hello, World!' in the file:

```
fs.open('example.txt', 'w', (err, fd) => {
  if (err) throw err;
  fs.write(fd, 'Hello, World!', (err) => {
    if (err) throw err;
    fs.close(fd, (err) => {
      if (err) throw err;
      console.log('File has been created');
    });
  });});
```

Creating and removing directories

The **fs.mkdir() method** is an inbuilt application programming interface of fs module which provides an API for interacting with the file system in a manner closely modeled around standard **POSIX** functions.

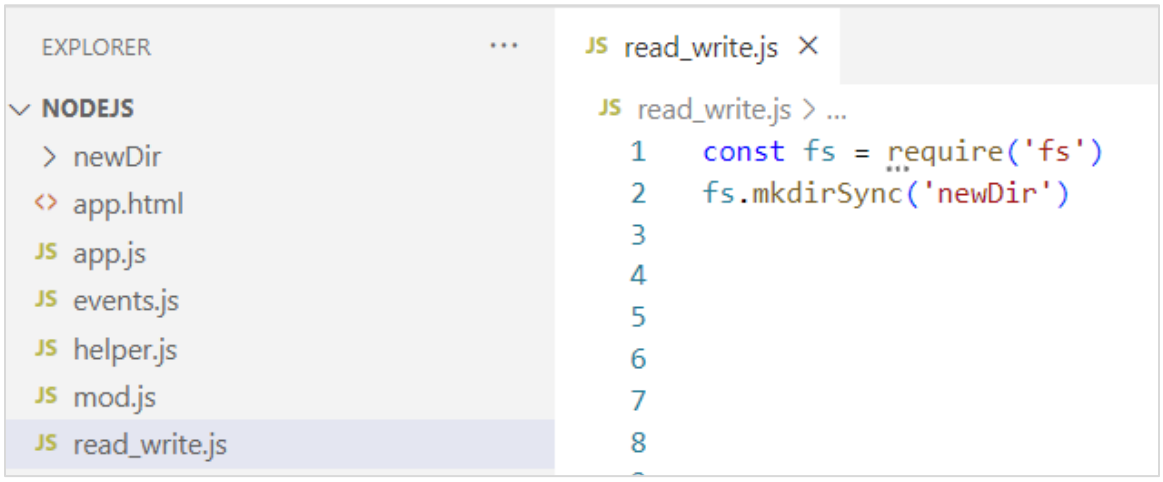
The **fs.mkdirSync() method** is used to create a directory Synchronously.

Syntax:
`fs.mkdirSync( path, options )`

Parameters: This method accept two parameters as mentioned above and described below:

- **path:** The path at which directory is to be created. It can be string, buffer, etc.
- **options:** It is an optional parameter which determines how to create directory like recursively, etc.

Example 15)



The **fs.mkdir()** method i Node.js is used to create a directory asynchronously.

Syntax
`fs.mkdir(path, mode, callback)`

Parameters: This method accept three parameters as mentioned above and described below:

- **path:** This parameter holds the path of the directory must be created.
- **mode:** This parameter holds the recursive boolean value. The mode option is used to set the directory permission, by default it is 0777.
- **callback:** This parameter holds the callback function that contains error. The recursive option if set to true will not give an error message if the directory to be created already exists.

Example 16)

```
const fs = require('fs')
const path = require('path');

fs.mkdir(path.join(__dirname, 'new_dir'), (err) => {
    if (err) {
        return console.error(err);
    }
    console.log('Directory created successfully!');
});
```

6.4. Clients and servers connection using nodeJS

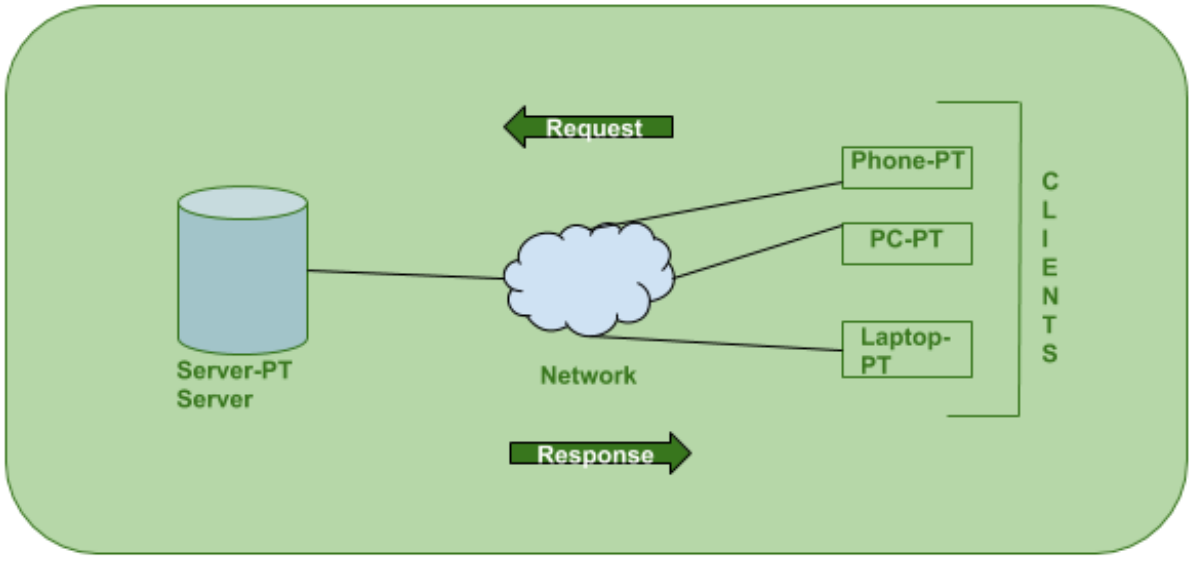
The Client-server model is a distributed application structure that partitions task or workload between the providers of a resource or service, called servers, and service requesters called clients. In the client-server architecture, when the client computer sends a request for data to the server through the internet, the server accepts the requested process and delivers the data packets requested back to the client. Clients do not share any of their resources. Examples of Client-Server Model are Email, World Wide Web, etc.

6.4.1. How the Client-Server Model works ?

In this session we are going to take a dive into the **Client-Server** model and have a look at how the **Internet** works via, web browsers. This session will help us in having a solid foundation of the WEB and help in working with WEB technologies with ease.

- **Client:** When we talk the word **Client**, it means to talk of a person or an organization using a particular service. Similarly in the digital world a **Client** is a computer (**Host**) i.e. capable of receiving information or using a particular service from the service providers (**Servers**).
- **Servers:** Similarly, when we talk the word **Servers**, it means a person or medium that serves something. Similarly in this digital world a **Server** is a remote computer which provides information (data) or access to particular services.

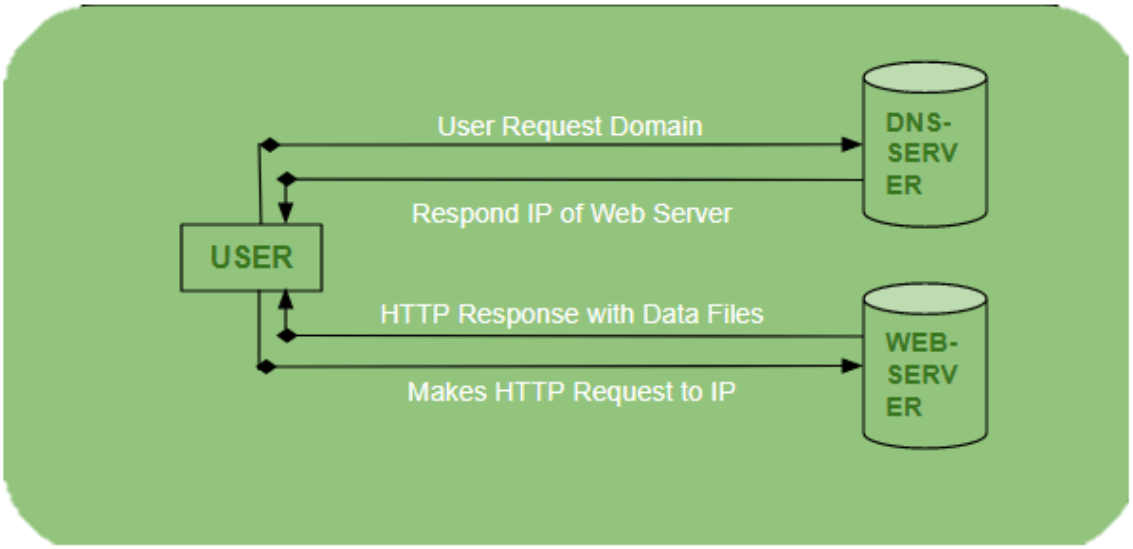
So, its basically the **Client** requesting something and the **Server** serving it as long as its present in the database.



How the browser interacts with the servers ?

There are few steps to follow to interacts with the servers a client.

- User enters the **URL**(Uniform Resource Locator) of the website or file. The Browser then requests the **DNS**(DOMAIN NAME SYSTEM) Server.
- **DNS Server** lookup for the address of the **WEB Server**.
- **DNS Server** responds with the **IP address** of the **WEB Server**.
- Browser sends over an **HTTP/HTTPS** request to **WEB Server's IP** (provided by **DNS server**).
- Server sends over the necessary files of the website.
- Browser then renders the files and the website is displayed. This rendering is done with the help of **DOM** (Document Object Model) interpreter, **CSS** interpreter and **JS Engine** collectively known as the **JIT** or (Just in Time) Compilers.



Advantages of Client-Server model:

- Centralized system with all data in a single place.
- Cost efficient requires less maintenance cost and Data recovery is possible.
- The capacity of the Client and Servers can be changed separately.

Disadvantages of Client-Server model:

- Clients are prone to viruses, Trojans and worms if present in the Server or uploaded into the Server.
- Server are prone to Denial of Service (DOS) attacks.

- Data packets may be spoofed or modified during transmission.
- Phishing or capturing login credentials or other useful information of the user are common and MITM(Man in the Middle) attacks are common.

6.4.2. How to create a server in nodejs

Node.js is an open-source and cross-platform runtime environment for executing JavaScript code outside a browser. You need to remember that **NodeJS is not a framework, and it's not a programming language.** Node.js is mostly used in server-side programming.

Creating Web Servers Using NodeJS

There are mainly two ways to create a server in *nodejs* as follows.

1. Using http inbuilt module
2. Using express third party module

Using http module: HTTP and HTTPS, these two inbuilt modules are used to create a simple server. The HTTPS module provides the feature of the encryption of communication with the help of the secure layer feature of this module. Whereas the HTTP module doesn't provide the encryption of the data. (AAyush, 2023)

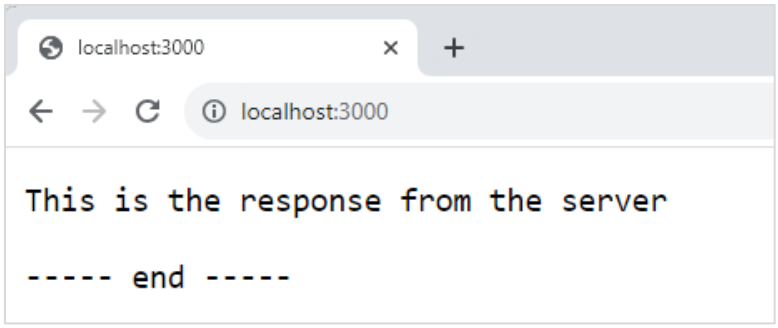
Example 17) Create a server using in *nodejs*

App.js

```
// import the http module
const http = require("http")
// create server
const server = http.createServer((request, response) =>{
    //send the request
    response.write("This is the response from the server")
    response.end("\n-------end-------")
    console.log(request.url)
})

// server listening to port 3000
server.listen((3000),function(){
    console.log("Server is running")
})
```

In the browser, URL the localhost:3000



6.4.3. Serving HTML

We can also create a server to listen to a HTML file. Let's create a *HTML* file:

index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-
scale=1.0">
    <title>Intro to NodeJS</title>
</head>
<style>
    body{background-color: darkblue;}
    .title{color:crimson; text-align: center;}
    .author{text-align: center; font-weight: bolder; color:
white;}
</style>
<body>
    <h1 class="title">Welcome to by Cyber World</h1>
    <h2 class="author">Student's full name</h2>

</body>
</html>
```

App.js

```
// import the http module
const http = require("http")
const fs = require("fs") // file streaming
```

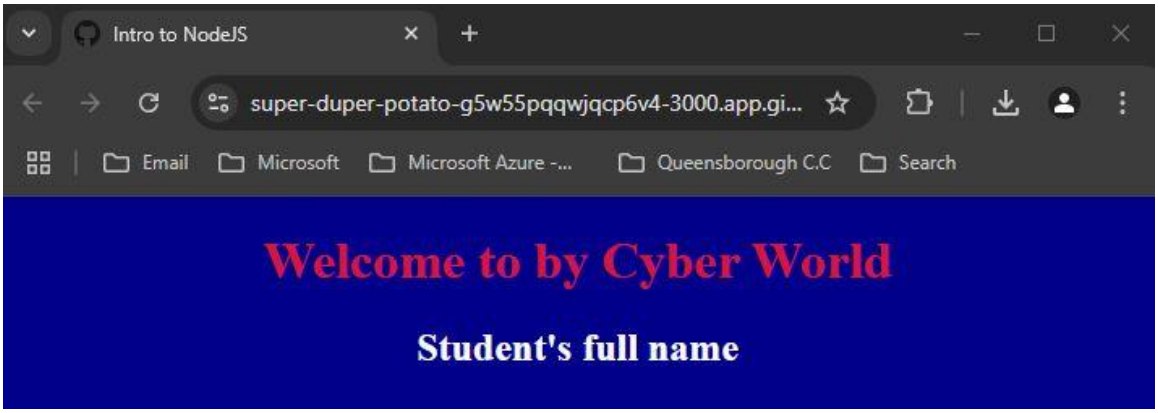
```
// create server
const server = http.createServer((request, response) =>{
    // send the request through the index.html
    response.writeHead(200, {'content-type': 'text/html'})
    const readstream = fs.createReadStream(__dirname +
'/index.html')

    //Piping
    readstream.pipe(response)
})

// server listening to port 3000
server.listen((3000),function(){
    console.log("Server is running")
})
```

After it, in the ***app.js*** file, we modify the `'Content-Type'` to **`'text/html'`** and change the path to `index.html`

Running the server will show:



6.3.4. Serving JSON data

To run a JSON data in a server, we first modify the `'Content-Type'` to **`'application/json'`**

Example 18) create a nodejs server that serves a JSON file

App.js

```
// import the http module
```

```
const http = require("http")
const fs = require("fs") // file streaming

// create server
const server = http.createServer((request, response) =>{
    // SEND THE REQUEST THROUGH A JSON FILE
    response.writeHead(200, {'content-type':'application/json'})
    // JSON data
    const user = {
        name: "Peter Pan",
        email:"peterpan@neverland.com"
    }
    // piping
    response.end(JSON.stringify(user))
})

// server listening to port 3000
server.listen((3000),function(){
    console.log("Server is running…")
})
```

6.4.5. Routing in node.js

Routing defines the way in which the client requests are handled by the application endpoints.

Implementation of routing in Node.js: There are two ways to implement routing in node.js which are listed below:

- By Using Framework
- Without using Framework

Using Framework: Node has many frameworks to help you to get your server up and running. The most popular is Express.js.

Routing with Express in Node: Express.js has an "app" object corresponding to HTTP. We define the routes by using the methods of this "app" object. This app object specifies a callback function called when a request is received. We have different methods of in-app objects for different types of requests. (Shuhamsh2820, 2023)

For a GET request using the **app.get()** method:

```
const express = require('express')
const app = express()
```

```
app.get('/', function(req, res) {
    res.send('Hello Sir')
})
```

For POST requests use the **app.post()** method:

```
const express = require('express')
const app = express()

app.post('/', function(req, res) {
    res.send('Hello Sir')
})
```

For handling all HTTP methods (i.e. GET, POST, PUT, DELETE, etc.) use the **app.all()** method:

```
const express = require('express')
const app = express()

app.all('/', function(req, res) {
    console.log('Hello Sir')
    next()   // Pass the control to the next handler
})
```

The next() is used to hand off the control to the next callback. Sometimes we use app.use() to specify the middleware function as the callback. So, to perform routing with Express.js you have only to load the express and then use the app object to handle the callbacks according to the requirement.

Routing without Framework: Using the frameworks is good to save time, but sometimes this may not suit the situation. So, a developer may need to build up their own server without other dependencies. Now create a file with any name using .js extension and follow the steps to perform routing from scratch:

Here we will use the built-in module of node.js i.e. HTTP. So, First load http:

```
const http = require('http');
```

Now create a server by adding the following lines of code:

```
http.createServer(function (req, res) {
    res.write('Hello World!'); // Write a response
    res.end(); // End the response
}).listen(3000, function() {
    // The server object listens on port 3000
    console.log("server start at port 3000");
});
```

Now add the following lines of code in the above function to perform routing:

```
const url = req.url;
if(url ==='/about') {
    res.write(' Welcome to about us page');
    res.end();
 } else if(url ==='/contact') {
    res.write(' Welcome to contact us page');
    res.end();
 } else {
    res.write('Hello World!');
    res.end();
 }
```

Example 19) The complete code of routing by combining the above code.

```
const http = require('http');

// Create a server object
http.createServer(function (req, res) {

        // http header
        res.writeHead(200, {'Content-Type': 'text/html'});

        const url = req.url;

        if(url ==='/about') {
                res.write(' Welcome to about us page');
                res.end();
        }
        else if(url ==='/contact') {
                res.write(' Welcome to contact us page');
                res.end();
        }
        else {
                res.write('Hello World!');
                res.end(); }
```

```
}).listen(3000, function() {

        // The server object listens on port 3000
        console.log("server start at port 3000");
});
```

Example 20) routing to a different html page in nodejs

For this example, let's create four more files: routing.js, index.html, about.html, and 404.html. We are going to use the *routing.js* file to route the three html pages.

The *index.html* file will be as:

index.html routing.js 404.html about.html

index.html > html > body

```
<!DOCTYPE html>
<html lang="en">

<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Type your name</title>
    <style>
        body {
            background-color: rgb(250, 250, 250)
        }

        .title {
            color: crimson;
            text-align: center;
            font-size: 3em;
        }
    </style>
</head>

<body>
    <div class="container">
        <h1 class="title">Welcome to my Cyber World!</h1>
    </div>

</body>

</html>
```

The *about.html* file will be as:

index.html | routing.js | 404.html | about.html

about.html > html

```
<!DOCTYPE html>
<html lang="en">

<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>About us</title>
</head>
<style>
    body {
        background-color: azure;
    }

    .title {
        text-align: center;
        color: blueviolet;
        font-size: 2.5em;
    }
</style>

<body>
    <h1 class="title">About nodejs</h1>
</body>

</html>
```

The *404.html* file will be:

index.html | routing.js | 404.html | about.html

404.html > html

```
<!DOCTYPE html>
<html lang="en">

<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>404 error</title>
    <style>
        body {
            background-color: lightcoral;
        }

        .title {
            font-size: 2.5em;
            text-align: center;
        }
    </style>
</head>

<body>
    <h1 class="title">404 error: page not found!</h1>
</body>

</html>
```

The *routing.js* fill be:

routing.js > ...

```
1   const http = require('http');
2   const fs = require('fs')
3
4   const server = http.createServer((req,res) =>{
5
6       res.writeHead(200, {'Content-Type': 'text/html'});
7       const url = req.url;
8
9       if(url ==='/home' || req.url ==='/'){
10          fs.createReadStream('index.html').pipe(res)
11      }
12      else if (url==='/about'){
13          fs.createReadStream('about.html').pipe(res)
14      }
15      else{
16          res.write('Page does not exist! please go back home')
17      }
18  })
19  server.listen((3200), () => {
20      console.log('serve is running...')
21  })
```

References

AAyush, Z. (2023). *How to Build a Simple Web Server with Node.js ?* Retrieved from Geeks for Geeks: https://www.geeksforgeeks.org/how-to-build-a-simple-web-server-with-node-js/

Arnubop. (2023). *Node.js fs.readFileSync() Method*. Retrieved from DSA tutorial: https://www.geeksforgeeks.org/node-js-fs-readfilesync-method/

Atta. (2023). *How to read and write files in Node.js*. Retrieved from atta blog: https://attacomsian.com/blog/reading-writing-files-nodejs

Educative. (2023). *What is Node.js? A beginner's introduction to JavaScript runtime*. Retrieved from Educative: https://www.educative.io/blog/what-is-nodejs

Geeks, G. f. (2023). *What does double underscore (__) in front of a variable in Node.js ?* Retrieved from Geeks for Geeks: https://www.geeksforgeeks.org/what-does-double-underscore-__-in-front-of-a-variable-in-node-js/

moonpatel2003. (2023). *What is piping in node.js?* Retrieved from Geeks for Geeks: https://www.geeksforgeeks.org/what-is-piping-in-node-js/

nitrin_sharma. (n.d.). *Node.js fs.createWriteStream() Method*. Retrieved from Geeks for Geeks: https://www.geeksforgeeks.org/node-js-fs-createwritestream-method/

org, N. (2023). *NodeJS org*. Retrieved from About NodeJS .

Shuhamsh2820. (2023, March). *Routing in Node.js*. Retrieved from Geeks for Geeks: https://www.geeksforgeeks.org/routing-in-node-js/

Tandon, K. (2019). *Streams and Buffers in Node.js*. Retrieved from Medium: https://medium.com/developers-arena/streams-and-buffers-in-nodejs-30ff53edd50f

w3schools. (n.d.). *Node.js*. Retrieved from w3 school: https://www.w3schools.com/nodejs/nodejs_intro.asp